Python Engineering - Class 08 Reference Guide

Data Processing

Reference guide to the syntax and tools used in this class. It explains what each element does, how it is written and when it is useful.

Quick map

Concept	Syntax	Idea
Boolean mask	mask = h > 100	Array of True/False values aligned with data.
&	(h>50) & (v>0)	Element-wise logical AND for NumPy conditions.
|	cond1 | cond2	Element-wise logical OR.
~	~mask	Element-wise Boolean negation.
A[mask]	trajectory[mask]	Select rows/elements where mask is True.
np.sum(mask)	np.sum(mask)	Count True values because True behaves like 1.
np.where()	np.where(mask)[0]	Return indices where condition is True.
np.mean()/std()	np.mean(x), np.std(x)	Basic descriptive statistics.
np.diff()	np.diff(t)	Difference between consecutive samples; output length N-1.
np.allclose()	np.allclose(dt_samples,dt)	Tolerance-based numerical comparison.
np.column_stack()	np.column_stack((A,energy))	Join 1D columns into a 2D matrix.

1. Boolean mask

Syntax: `mask = h > 100`

Array of True/False values aligned with data.

```
mask = heights > 100
```

Control question: before running the example, predict the result type and shape.

2. &

Syntax: `(h>50) & (v>0)`

Element-wise logical AND for NumPy conditions.

```
mask = (heights > 50) & (velocities > 0)
```

Control question: before running the example, predict the result type and shape.

3. |

Syntax: `cond1 | cond2`

Element-wise logical OR.

```
mask = (t < 2) | (t > 5)
```

Control question: before running the example, predict the result type and shape.

4. ~

Syntax: `~mask`

Element-wise Boolean negation.

```
not_ascending = ~(velocities > 0)
```

Control question: before running the example, predict the result type and shape.

5. A[mask]

Syntax: `trajectory[mask]`

Select rows/elements where mask is True.

```
selected_rows = trajectory[mask]
```

Control question: before running the example, predict the result type and shape.

6. np.sum(mask)

Syntax: `np.sum(mask)`

Count True values because True behaves like 1.

```
n_ascending = np.sum(velocities > 0)
```

Control question: before running the example, predict the result type and shape.

7. np.where()

Syntax: `np.where(mask)[0]`

Return indices where condition is True.

```
indices = np.where(heights > 100)[0]
```

Control question: before running the example, predict the result type and shape.

8. np.mean()/std()

Syntax: `np.mean(x), np.std(x)`

Basic descriptive statistics.

```
mean_h = np.mean(heights)
std_h = np.std(heights)
```

Control question: before running the example, predict the result type and shape.

9. np.diff()

Syntax: `np.diff(t)`

Difference between consecutive samples; output length N-1.

```
dt_samples = np.diff(times)
```

Control question: before running the example, predict the result type and shape.

10. np.allclose()

Syntax: `np.allclose(dt_samples,dt)`

Tolerance-based numerical comparison.

```
constant_dt = np.allclose(np.diff(times), dt)
```

Control question: before running the example, predict the result type and shape.

11. np.column_stack()

Syntax: `np.column_stack((A,energy))`

Join 1D columns into a 2D matrix.

```
processed = np.column_stack((trajectory, energy))
```

Control question: before running the example, predict the result type and shape.

Common mistakes

- Confusing a function name with a function call using parentheses.
- Not checking types, indices or shapes before an operation.
- Using a tool because it works without being able to explain what it receives and returns.

Mental cheat sheet

```
Boolean mask: mask = h > 100
&: (h>50) & (v>0)
|: cond1 | cond2
~: ~mask
A[mask]: trajectory[mask]
np.sum(mask): np.sum(mask)
np.where(): np.where(mask)[0]
np.mean()/std(): np.mean(x), np.std(x)
np.diff(): np.diff(t)
np.allclose(): np.allclose(dt_samples,dt)
np.column_stack(): np.column_stack((A,energy))
```